Algorithm Design and Analysis (Spring 2026)

P1

(15 points) Prove the following generalization of the master theorem. Given constants $a \geq 1, b > 1, d \geq 0$, and $w \geq 0$, if $T(n) = 1$ for $n < b$ and $T(n) = aT(n/b) + n^d \log^w n$, we have

$$T(n) = \begin{cases} O(n^d \log^w n), & \text{if } a < b^d, \\ O(n^{\log_b a}), & \text{if } a > b^d, \\ O(n^d \log^{w+1} n), & \text{if } a = b^d. \end{cases}$$

Prove by drawing a table:

level	num of problems	problem scale	level complexity
0	1	n	$n^d \log^w n$
1	a	$\frac{n}{b}$	$a \cdot \left(\frac{n}{b}\right)^d \log^w \left(\frac{n}{b}\right)$
2	a^2	$\frac{n}{b^2}$	$a^2 \cdot \left(\frac{n}{b^2}\right)^d \log^w \left(\frac{n}{b^2}\right)$
k	a^k	$\frac{n}{b^k}$	$a^k \cdot \left(\frac{n}{b^k}\right)^d \log^w \left(\frac{n}{b^k}\right)$
$\cdots$			
$\log_b n$	$a^{\log_b n}$	1	$a^{\log_b n}$

The total running time is

$$n^d \log^w n + a \cdot \left(\frac{n}{b}\right)^d \log^w \left(\frac{n}{b}\right) + \cdots + a^k \cdot \left(\frac{n}{b^k}\right)^d \log^w \left(\frac{n}{b^k}\right) + \cdots + a^{\log_b n}$$

Simplification

$$n^d \cdot \left(\log^w n + \frac{a}{b^d} \log^w \left(\frac{n}{b}\right) + \cdots \left(\frac{a}{b^d}\right)^k \log^w \left(\frac{n}{b^k}\right) + \cdots + \left(\frac{a}{b^d}\right)^{\log_b n}\right)$$

Convert into sum expression

$$T(n) = n^d \cdot \left(\sum_{k=0}^{\log_b n} \left(\frac{a}{b^d}\right)^k (\log n - k \log b)^w\right) + n^{\log_b a}$$

> Note: When $k = \log_b n$, $\left(\log \frac{n}{b^k}\right)^w = 0$, the leaf cost vanishes. But $T(1)$ is actually 1. So we must add the last term $a^{\log_b n} = a^{\frac{\log_a n}{\log_a b}} = n^{\log_b a}$ separately.

- Case 1: $a < b^d$

$$T(n) < n^d \log^w n \cdot \sum_{k=0}^{\log_b n} \left(\frac{a}{b^d}\right)^k + n^{\log_b a}$$

Given $\frac{a}{b^d} < 1$,

$$\sum_{k=0}^{\log_b n} \left(\frac{a}{b^d}\right)^k < \sum_{k=0}^{\infty} \left(\frac{a}{b^d}\right)^k = O(1)$$

So

$$T(n) = O(n^d \log^w n + n^{\log_b a})$$

We also have $\log_b a < d$. So $n^{\log_b a} < n^d$. Thus

$$T(n) = O(n^d \log^w n)$$

- Case 2: $a > b^d$: In this case $\frac{a}{b^d} > 1$. The last term dominates the total complexity. We set $j = \log_b n - k$

$$\begin{aligned}
T(n) &= n^d \cdot \left(\sum_{k=0}^{\log_b n} \left(\frac{a}{b^d}\right)^k (\log n - k \log b)^w\right) + n^{\log_b a} \\
&= n^d \cdot \left(\sum_{j=0}^{\log_b n} \left(\frac{a}{b^d}\right)^{\log_b n - j} (j \log b)^w\right) + n^{\log_b a} \\
&= n^{\log_b a} \cdot \left(\sum_{j=0}^{\log_b n} \left(\frac{a}{b^d}\right)^{-j} (j \log b)^w\right) + n^{\log_b a} \\
&= n^{\log_b a} \cdot \left(\sum_{j=0}^{\log_b n} \left(\frac{a}{b^d}\right)^{-j} (j \log b)^w + 1\right)
\end{aligned}$$

Let's consider the summation term. For the sake of simplification, we denote a/b^d as r, and ignore the constant coefficient $\log^w b$.

$$\sum_{j=0}^{\log_b n} \left(\frac{a}{b^d}\right)^{-j} (j \log b)^w \propto \sum_{j=0}^{\log_b n} r^{-j} j^w < \sum_{j=0}^{\infty} r^{-j} j^w$$

Since $r < 1$, the summation term consists of an exponential decay and a polynomial increase. We know the sum converges. i.e. $\exists C \in \mathbb{R}$, s.t. $\sum_{j=0}^{\infty} r^{-j} j^w < C$. So the huge scaring summation factor is actually $O(1)$. We can now conclude that

$$T(n) = O(n^{\log_b a})$$

- Case 3: $a = b^d$:

$$\begin{aligned}
T(n) &= n^d \cdot \sum_{k=0}^{\log_b n} (\log n - k \log b)^w + n^d \\
&< n^d \cdot \log_b n \cdot \log^w n + n^d \\
&= \frac{1}{\log b} n^d \log^{w+1} n + n^d \\
&= O(n^d \log^{w+1} n)
\end{aligned}$$

P2

(25 points) Consider the randomized quicksort algorithm, where each pivot is chosen uniformly at random. Analyze its expected running time.

a. (10 points) Prove that the expected running time is $O(n^c)$ for some constant $c < 2$, using a method similar to the one presented in lecture. (You may choose any constant $c < 2$.) In particular, define a suitable "lucky" event, and analyze the lucky and unlucky cases separately.

Proof: We call a pivot p *lucky* if $p \in [\frac{n}{3}, \frac{2n}{3}]$. If p is not lucky, then it is *unlucky*.

The recursive form of the compexity of quicksort is

$$T(n) = T(L) + T(R) + O(n)$$

Where $L + R = n - 1$.

- Case 1: The pivot is lucky

  $$\max(L, R) \leq \frac{2n}{3}$$

  In this case,

  $$T(n) \leq 2T\left(\frac{2n}{3}\right) + O(n)$$

- Case 2: The pivot is unlucky

  We consider the worst case

  $$T(n) \leq T(n-1) + O(n)$$

We assume the pivot p is uniformly distributed. So $P(p \text{ is lucky}) = \frac{1}{3}$. Let $\tau(n)$ be the total cost we take until the first lucky pivot occurs. Since we are lucky with the probability $\frac{1}{3}$, the depth of our recursion before we get a lucky pivot follows a geometric distribution with expectation 3. So $E(\tau(n)) = O(n)$

$$T(n) = \tau(n) + 2T(\tfrac{2n}{3})$$

We take expectation from both side

$$E(T(n)) = 2E(T(\tfrac{2n}{3})) + O(n)$$

Refer to the master theorem, $E(T(n)) = O\left(n^{\log_{\frac{3}{2}} 2}\right) = O(n^{1.71})$. We conclude

$$E(T(n)) = O(n^c), \quad c = 1.71 < 2$$

b. (15 points) Prove that the expected running time is $O(n \log n)$. Hint: Let x_i be the i -th smallest element and x_j the j -th smallest element, where $j > i$. Show that x_i and x_j are compared in quicksort if and only if either x_i or x_j is the first pivot chosen among the elements $x_i, x_{i+1}, \ldots, x_j$. What is the probability of this event?

Proof: Let x_i and x_j be the i-th and j-th smallest element relatively $(i < j)$. Define indicator variable:

$$X_{ij} = \begin{cases} 1 & \text{if } x_i \text{ and } x_j \text{ are compared} \\ 0 & \text{otherwise} \end{cases}$$

Then the total comparisons:

$$X = \sum_{i<j} X_{ij}$$

The expectation:

$$E(X) = \sum_{i<j} E(X_i j) = \sum_{i<j} P(x_i \text{ and } x_j \text{ are compared})$$

We claim that x_j and x_j are compared in quicksort iff either x_i or x_j is the first pivot chosen among $x_i, x_{i+1}, \cdots, x_j$. In fact, if some x_k $(i < k < j)$ is chosen first as a pivot, then x_i and x_j are divided into different sub questions, and are never compared. On the other hand, if x_i or x_j is chosen as a pivot, then x_i and x_j must be compared.

Since the pivot is chosen uniformly:

$$P(x_i \text{ and } x_j \text{ are compared}) = \frac{2}{j-i+1}$$

Thus

$$E(X) = \sum_{i<j} \frac{2}{j-i+1}$$

Let $k = j - i$, then

$$E(X) = \sum_{k=1}^{n-1} (n-k) \cdot \frac{2}{k+1} \leq \sum_{k=1}^{n} \frac{2}{k+1} \cdot n = O(n \log n)$$

Therefore:

$$E(T(n)) = O(n \log n)$$

P3

(30 points) Given an $n \times m$ 2-dimensional integer array $A|0, \ldots, n-1; 0, \ldots, m-1|$ where $A[i, j]$ denotes the cell at the i -th row and the j -th column, a local minimum is a cell $A[i, j]$ such that $A[i, j]$ is smaller than each of its four adjacent cells $A[i-1, j], A[i+1, j], A[i, j-1], A[i, j+1]$. Notice that $A[i, j]$ only has three adjacent cells if it is on the boundary, and it only has two adjacent cells if it is at the corner. Assume all the cells have distinct values. Your objective is to find one local minimum (i.e., you do not have to find all of them).

a. (10 points) Suppose $m = 1$ so A is a 1-dimensional array. Design a divide-andconquer-based algorithm for the problem above. Write a recurrence relation of the algorithm, and analyze its running time.

Algorithm

First check the position `mid = (left+right)/2` and its neighbors.

- case 1: `mid` is a local minimum if $A[\text{mid}] < A[\text{mid}-1]$ 且 $A[\text{mid}] < A[\text{mid}+1]$ then return mid
- case 2: The left neighbor is smaller if $A[\text{mid}-1] < A[\text{mid}]$, there exists a local min in the left half of the array. So recursively search left half of the array.
- case 3: The right neighbor is smaller Symmetrical to case 2, recursively search the right hand of the array.

Pseudo code:

```
def LocalMin(A, left, right):
  """
  Args:
      A: the array to find a local minimum
      left: the left bound of searching interval
      right: the right bound of searching interval
  Returns:
      a local minimum in A[left: right]
  """
  if (left == right):
      return left

  mid = (left + right) // 2

  # handle boundary situation
  left_val = A[mid-1] if mid > left else INF
  right_val = A[mid+1] if mid < right else INF

  if (A[mid] < left_val and A[mid] < right_val):
      return mid
  elif (left_val < A[mid]):
      return LocalMin(A, left, mid-1)
  else:
      return LocalMin(A, mid+1, right)
```

Validity analysis

We need to show that if $A[\text{mid}-1] < A[\text{mid}]$, then there exists a local minimum $A[k]$ s.t. $\text{left} \leq k \leq \text{mid}-1$.

If the subsequence is strictly increasing, we claim $A[\text{left}]$ is a local minimum. In fact, if $\text{left} = 0$, in other word $A[\text{left}]$ is in the on the boundary, by definition it is a local minimum. If $\text{left} \neq 0$, by recursive hypothesis, $A[\text{left}] < A[\text{left}-1]$. The increase of the sequence implies that $A[\text{left}] < A[\text{left}+1]$. So $A[\text{left}]$ is a local minimum.

If the subsequence is not strictly increasing, then there exists an index k such that $A[k] < A[k-1]$ and $A[k] < A[k+1]$. In particular, when the subsequence is strictly decreasing, $k = \text{mid} - 1$.

Hence, a local minimum must exist in the left half.

Complexity analysis

$$T(n) = T(\tfrac{n}{2}) + O(1)$$

By master theorem

$$T(n) = O(\log n)$$

---

b. (10 points) Suppose $m = n$. Design a divide-and-conquer-based algorithm for the problem above. Write a recurrence relation of the algorithm, and analyze its running time.

Algorithm

We implement a divide-and-conquer approach by performing a binary search on the rows.

1. Pick the middle row `mid_row = (upper + lower) // 2`.
2. Find the global minimum of this row. Let the minimum be at $A[\text{mid_row}][i]$, where $0 \leq i \leq n-1$.
3. Compare $A[\text{mid_row}][i]$ with its vertical neighbors $A[\text{mid_row} - 1][i]$ and $A[\text{mid_row} + 1][i]$ (if exist)
    - case 1: $A[\text{mid_row}][i]$ is smaller than their neighbors, then $A[\text{mid_row}][i]$ is already a local minimum. Return it.
    - case 2: $A[\text{mid_row} - 1][i] < A[\text{mid_row}][i]$, there exist a local minimum in the upper half of the matrix. Recursively search the upper half sub-matrix.
    - case 3: $A[\text{mid_row} + 1][i] < A[\text{mid_row}][i]$, symmetric to case 2, recursively search the lower half sub-matrix.

Pseudo code

```python
def LocalMin2D(A, upper, lower):
    """
    Args:
        A: The n * n 2D array
        upper, lower: The row boundaries for the current search.
    Returns:
        The coordinates (i, j) of a local minimum
    """
    if (upper == lower):
        # obtain the column num of the minimum in this row
        min_column = argmin(k, A[upper][k]) # O(n) complexity
        return (upper, min_column)

    mid_row = (upper + lower) // 2

    min_column = argmin(k, A[mid_row][k]) # O(n) complexity
    current_val = A[mid_row][min_column]
    # handle boundary situations
    upper_val = A[mid_row-1][min_column] if mid_row > upper else INF
    lower_val = A[mid_row+1][min_column] if mid_row < lower else INF

    if (current_val < upper_val and current_val < lower_val):
        return (mid_row, min_column)
    elif (upper_val < current_val):
        return LocalMin2D(A, upper, mid_row-1)
    else:
        return LocalMin2D(A, mid_row+1, lower)
```

Validity analysis

If the recursion returns **before** reach the base case, where $\text{upper} = \text{lower}$, it is trival that the return value is a local minimum.

If the recursion reaches the base case, we need to check whether the return value is smaller than its vertical neighbors.

In fact, when we reach the base case, by recursion hypothesis, the value of some column in the row is smaller than the minimum of its two neighbor rows (if exists). That's why we come to this specific row. Further more, the value of the returned coordinate is even smaller than (or equal to) that of the "some column" described before. So the value of the returned coordinate is smaller than its vertical neighbors.

The above is my explaination, which is not necessarily formal and accurate. Here I paste a graph of Chat-GPT's insight:

> At each step, we select the minimum element in the middle row. Therefore, this element is smaller than its horizontal neighbors.
>
> If it is also smaller than its vertical neighbors, then it is a local minimum.

> Otherwise, suppose the upper neighbor is smaller. Then we claim that there exists a local minimum in the upper half.
>
> Starting from this smaller neighbor, we can follow a path of strictly decreasing values. Since all values are distinct and the matrix is finite, this path must terminate at a cell that is smaller than all of its neighbors, i.e., a local minimum.
>
> Hence, the recursion is correct.

Complexity analysis

The complexity of an $n \times m$ grid is $T(n, m)$ then

$$T(n, m) = T(n/2, m) + O(m)$$

The $O(n)$ complexity comes from the `argmin` function. There are in total $\log n$ levels of recursion before we reach the base case. Each level contributes an $O(m)$ complexity. Thus

$$T(n, m) = O(m \log n)$$

In this question, $m = n$, so

$$T(n) = O(n \log n)$$

---

c. (10 points) Generalize your algorithm such that it works for general m and n . The running time of your algorithm should smoothly interpolate between the running times for the first two parts.

We can naturally generalize the algorithm in (b). But consider the different rate at which m and n influence the total running time, we implement a trick here:

Algorithm

If $m > n$, run $\mathrm{LocalMin2D}(A^T)$.

Otherwise run $\mathrm{LocalMin2D}(A)$.

Complexity

$$T(n, m) = O(\min(m, n) \log(\max(m, n)))$$

P4

(30 points) We are given a n-vertex tournament graph $\vec{G} = (V, A)$, which is a directed version of a complete undirected graph $G = (V, E)$. For each undirected edge $e = (u, v) \in E$, we have a corresponding directed arc ($u \to v$ or $v \to u$) in A . We aim to design an efficient algorithm to find a directed Hamiltonian path in $\vec{G}$, which is a path including all vertices in V , such as $u_1 \to u_2 \to u_3 \cdots \to u_n$.

a. (15 points) Assume we have a path P with length k , and another vertex u not included in P . Prove that we can always find a feasible location in P to insert u . (E.g., when $P = v_1 \to v_2 \to$

v_3 , we can insert u into P and change P to $v_1 \to u \to v_2 \to v_3$ if $v_1 \to u$ and $u \to v_2$ are both in A .) Design an $O(k)$ algorithm to find a feasible location to insert u into P , analyze the running time, and prove the correctness.

Algorithm

Scan the path $P = v_1 \to \cdots \to v_k$, find the smallest index i such that $u \to v_i$. If no such i exists, append u at the end.

Validity analysis There are in total two cases:

- If i exists. Since the graph is a tournament, for all $j < i$, we must have $v_j \to u$. Hence:

$$v_{i-1} \to u \to v_i$$

  when $i > 1$. If $i = 1$, $u \to v_1$, which is trivial. So insertion is valid.

- Otherwise, $v_i \to u$ for all i. So appending u maintains a valid path.

Running time The algorithm scans the path once. So time complexity is

$$T(k) = O(k)$$

---

b. (15 points) Use divide and conquer to find a $O(n \log n)$ algorithm to find a directed Hamiltonian path, analyze the running time, and prove the correctness.

Algorithm Recursively split the vertex set into two halves, compute Hamiltonian paths P_1 and P_2, and merge them by repeatedly comparing the current heads of the two paths and appending the vertex whose outgoing edge points to the other.

Pseudo code

```
def merge(P1: List[Vertex], P2: List[Vertex]) -> List[Vertex]:
    i, j = 0, 0
    P = []
    while (i < len(P1) and j < len(P2)):
        if (P1[i].to(P2[j])):
            P.append(P1[i])
            i += 1
        else:
            P.append(P2[j])
            j += 1

    # Append the rest
    while i < len(P1):
        P.append(P1[i])
        i += 1

    while j < len(P2):
        P.append(P2[j])
        j += 1
    return P
```

```python
def HamiltonianPath(V: List[Vertex]) -> List[Vertex]:
    """
    Args:
        V: Vertices of the tournament
    Returns:
        A Hamiltonian path of the tournament
    """
    if (len(V) == 1):
        return V

    size = len(V)
    V1 = V[:size//2]
    V2 = V[size//2:]

    P1 = HamiltonianPath(V1)
    P2 = HamiltonianPath(V2)

    return merge(P1, P2)
```

Validity analysis

We know the facts:

1. Each recursive call returns a valid Hamiltonian path.
2. During merge, for any two candidate vertices a_i and b_j, exactly one of $a_i \to b_j$ or $b_j \to a_i$ holds.
3. During merge, the algorithm always appends the vertex whose outgoing edge points to the other.

Let P be the path being constructed during the merge. Suppose at some step, x is the latest vertex in P, and y is the next vertex to be append.

- If x and y are from the same path. By fact 1, it is natural that $x \to y$.
- Otherwise, x and y are from different paths. If $y \to x$ holds, then in the previous step, where x and y are the two candidates, the algorithm chose x instead of y. This violates fact 3. By fact 2, $x \to y$ holds.

Thus every consecutive pair in the merged sequence satisfies the required direction, and the sequence contains all vertices. By induction, the final path is a directed Hamiltonian path of the whole tournament.

Running time

The recursive expression is

$$T(n) = 2T(\tfrac{n}{2}) + O(n)$$

Where $O(n)$ comes from the `merge` operation.

Thus

$$T(n) = O(n \log n)$$

P5

How long does it take you to finish the assignment (including thinking and discussion)? Give a score (1,2,3,4,5) to the difficulty. Do you have any collaborators? Please write down their names here.

Problem	Time spent	Difficulty	Dependence on LLM	Comment
1	一个下午	3	50%	没有 LLM 我就证不出来 Case2
2	半个晚上	2	50%	在宿舍写的，舍友叫我别卷（
3	3 节水课	4	70%	家人们 AI 真是太好用了
4	2 节水课	3	50%	终于写完了！耶
5	实时维护	0	0	